

一文详解模型权重存储新格式 Safetensors



AI代码助手上线啦

选中代码，体验AI替你一键

快速解读代码

立即体验

吃果冻不吐果冻皮 2024-07-01 3,049 阅读8分钟

六江

Trae 已上线 DeepSeek V3-0324，编程能力直逼 Claude 3.7 点击安装>>

觉得还不错?

一键收藏 以便后续温习~

社区首发签约文章，30天内禁止转载，30天后未获授权禁止转载，侵权必

在日常AI模型训练过程中，训练好的模型权重通常需要以一种格式存储在磁盘中。比如：目前最流行的AI框架 PyTorch 使用 pickle 格式存储模型权重文件。但 PyTorch 文档中有一段话说明如下：使用 `torch.load()` 保存模型，除非 `weights_only` 参数设置为True（只加载张量、原始类型和字典），否则隐式使用 `pickle` 模块，这是不安全的。可以构造恶意的 pickle 数据，该数据将在 `unpickling` 期间执行任意代码。切勿在不安全模式下加载可能来自不受信任来源的数据或可能已被篡改的数据，仅加载您信任的数据。为了规避类似的问题，新的权重存储格式 Safetensors 应运而生。

Safetensors 简介

Safetensors 是一种用于安全地存储张量的新格式，非常简单，但速度仍然很快（零拷贝）。它是 `pickle` 格式的替代品，因为，`pickle` 格式不安全，可能包含可以执行的恶意代码。

Safetensors 内部格式

假设现在有一个名为 `model.safetensors` 的Safetensors文件，那么 `model.safetensors` 内部格式如下：

登录掘金领取礼包

更多登录后权益等你解锁

登录 / 注册

File name **model.safetensors**

8 bytes N bytes rest of the file

N = u64 int containing
the size of the header

101101000110011

offsets: [BEGIN

AI代码助手上线啦

选中代码，体验AI替你一键
快速解读代码

立即体验

JSON utf-8 string representing the header

```
{
  "TENSOR_NAME_1": {
    "dtype": DATA_TYPE, // ex: "F16"
    "shape": List<Integer>, // ex: [1, 16, 256]
    "offsets": [BEGIN, END] // ex: [457, 8576]
  },
  "TENSOR_NAME_2": {...},
  ...,
  "__metadata__": {...} // special key for storing
                        free form text-to-text map
}
// DATA_TYPE can be one of ["F64", "F32", "F16", "BF16",
                             "I64", "I32", "I16", "I8", "U8", "BOOL"]
```

觉得还不错?

一键收藏 以便后续温习~

@稀土掘金技术社区

- 前面的 8 bytes 是一个无符号的整数，表示 header 占的字节数。
- 中间的 N bytes 是一个 UTF-8 编码 JSON 字符串，存储 header 的内容，里面为模型权重的元数据信息。
- 文件的剩余部分存储模型权重 tensor 的值。

以 GPT2 的 Safetensors 文件为例，其元数据信息可以通过如下代码获取：

python



复制代码

```
1 import requests # pip install requests
2 import struct
3
4 def parse_single_file(url):
5     # Fetch the first 8 bytes of the file
6     headers = {'Range': 'bytes=0-7'}
7     response = requests.get(url, headers=headers)
8     # Interpret the bytes as a little-endian
9     length_of_header = struct.unpack('<Q', response.content)[0]
10    # Fetch length_of_header bytes starting
11    headers = {'Range': f'bytes=8-{7 + length_of_header}'}
12    response = requests.get(url, headers=headers)
13    # Interpret the response as a JSON object
```



登录掘金领取礼包

更多登录后权益等你解锁



```

14     header = response.json()
15     return header
16
17 url = "https://huggingface.co/gpt2/resolve/main/model.safetensors"
18 header = parse_single_file(url)
19
20 print(header)
21 # {
22 #   "__metadata__": { "format": "pt" },
23 #   "h.10.ln_1.weight": {
24 #     "dtype": "F32",
25 #     "shape": [768],
26 #     "data_offsets": [223154176, 223157248]
27 #   },
28 #   ...
29 # }

```



AI代码助手上线啦

选中代码，体验AI替你一键
快速解读代码

立即体验

觉得还不错？

[一键收藏](#) 以便后续温习～

模型权重格式大比拼

对于不同模型权重格式，从如下几个方面进行全面的对比：

- 安全性：是否可以使用随机下载的文件并期望不运行任意代码吗？
- 零拷贝：读取文件是否需要比原始文件更多的内存？
- 延迟加载：可以在不加载所有内容的情况下检查文件吗？并仅加载其中的一些张量而不扫描整个文件吗？
- 布局控制：延迟加载不一定足够，因为如果有关张量的信息分散在您的文件中，那么即使可以延迟访问该信息，您也可能必须访问大部分文件才能读取可用的张量（导致许多磁盘到RAM的拷贝）。因此，控制布局以快速访问单个张量非常重要。
- 无文件大小限制：文件大小有限制吗？
- 灵活性：是否可以以该格式保存自定义代码并能够在以后以零额外代码使用它吗？（意味着我们可以存储不仅仅是纯张量，还有代码）
- 支持Bfloat16：该格式是否支持原生 bfloat16（意味着不需要奇怪的解决方法）？因为Bfloat16 数据格式在机器学习领域变得越来越重要。

下图展示了常见的存储格式的特性。



登录掘金领取礼包

更多登录后权益等你解锁



Format	Safe	Zero-copy	Lazy loading	No file size limit	Layout control	Flexibility	Bfloat16
pickle (PyTorch)	X	X	X	✓	X	✓	✓
H5 (Tensorflow)	✓	X	✓	✓	~	~	X
SavedModel (Tensorflow)	✓	X	X	✓	✓	 AI代码助手上线啦 选中代码，体验AI替你一键快速解读代码 立即体验	
MsgPack (flax)	✓	✓	X	✓	X		
Protobuf (ONNX)	✓	X	X	X	X		
Cap'n'Proto	✓	✓	~	✓	✓	~	X
Arrow	?	?	?	?	?	?	X
Numpy (npz)	✓	?	?	X	✓	X	X
pdparams (Paddle)	X	X	X	✓	X	✓	✓
	✓	✓	✓	✓	✓	X	✓

觉得还不错？
一键收藏 以便后续温习～

safetensors 和 ONNX 的不同

safetensors 和 ONNX 具有不同的用途。

safetensors 是一种简单、安全、快速的文件格式，用于存储和加载张量。它是 Python 的 pickle 实用程序的替代品，更加安全；而后者不安全，可能包含可以执行的恶意代码。

ONNX（开放神经网络交换）是一种用于表示深度学习模型的开放格式。它允许您用不同深度学习框架（例如：PyTorch、TensorFlow、Caffe2 等）加载模型以一种方式保存模型。这使得在不同框架之间共享模型变得更容易。

综上所述，safetensors 用于安全快速地存储和加载张量，而 ONNX 用于在不同深度学习框架之间共享模型。这同样适用于其他模型共享框架。

补充：零拷贝技术

零拷贝的主要任务就是避免CPU将数据从一块存储中拷贝到另一块存储，主要就是利用各种技术，避免让CPU做大量的数据拷贝任务，以此减少CPU的负担。或者借助其他的组件来完成简单的数据传输任务，让CPU解脱出来专注别的任务。

下面通过从磁盘读取数据并将其发送到套接字(socket)的方式，来模拟大多数Web 应用程序中经常发生)。为了完成此操作，内核



登录掘金领取礼包
更多登录后权益等你解锁

大多数

操作系统术语：

用户空间和内核空间是由操作系统分隔的两个虚拟内存区域， 以提供内存保护和硬件保护， 防止恶意或错误的软件行为。

用户空间是应用程序软件和一些驱动程序执行的内存区域。每个进程都有自己的虚拟内存空间中， 除非明确允许， 否则不能访问其他进程的虚拟内存空间。进程只能通过系统调用与内核交互， 系统调用是内核向用户空间



AI代码助手上线啦

选中代码， 体验AI替你一键

快速解读代码

立即体验

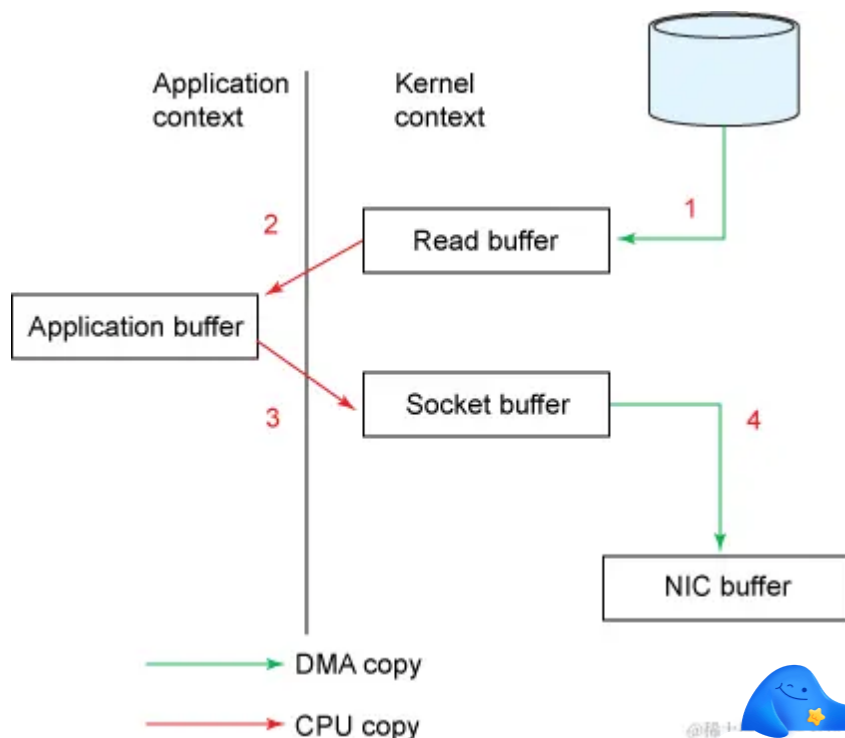
内核空间是操作系统内核、内核扩展和大多数设备驱动程序运行空间。设备驱动程序运行在内核模式下， 也称为管理模式， 这是一种特权模式， 允许访问所有CPU指令和硬件资源。

当数据从用户空间复制到内核空间， 它将再次执行内核调用， 内核会将数据写入套接字(socket)。每次数据从内核空间复制到用户空间， 都必须进行复制， 这会消耗 CPU 周期和内存带宽。

一键收藏 以便后续温习~

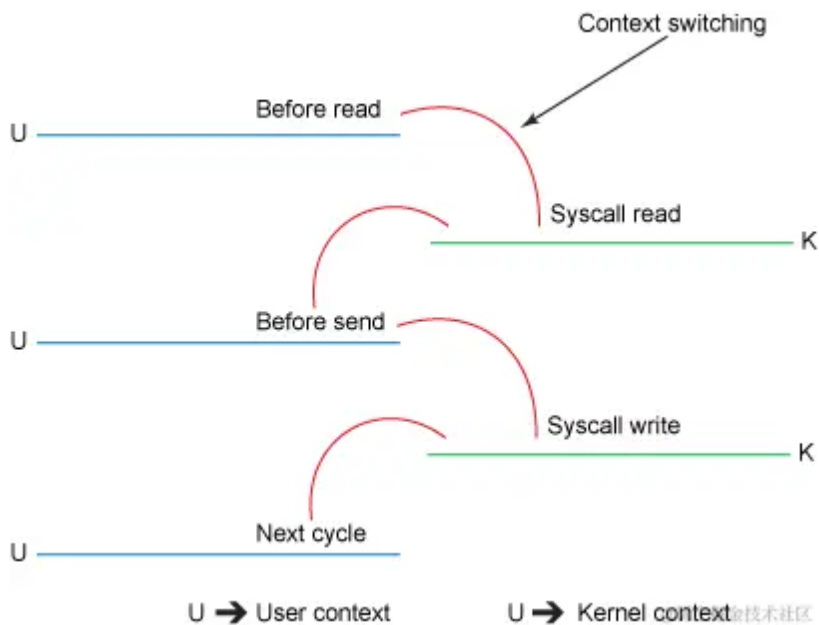
而零拷贝请求即内核直接将数据从磁盘文件复制到套接字， 而不经应用程序。

下图为传统的拷贝操作流程。除了拷贝操作之外， 这些系统调用会导致用户空间和内核空间之间发生大量上下文切换， 使得这个过程变慢。



登录掘金领取礼包

更多登录后权益等你解锁



AI代码助手上线啦

选中代码，体验AI替你一键

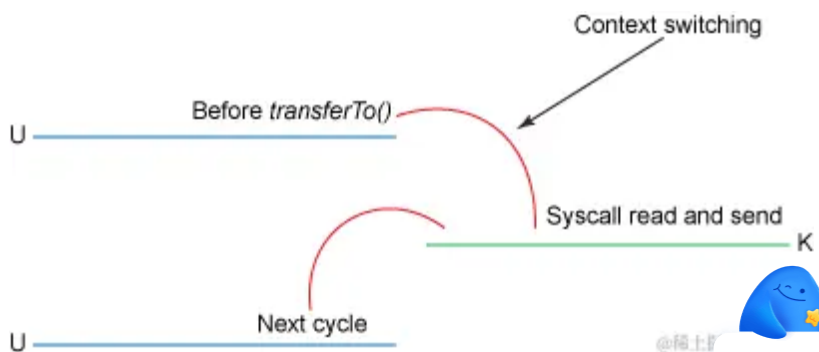
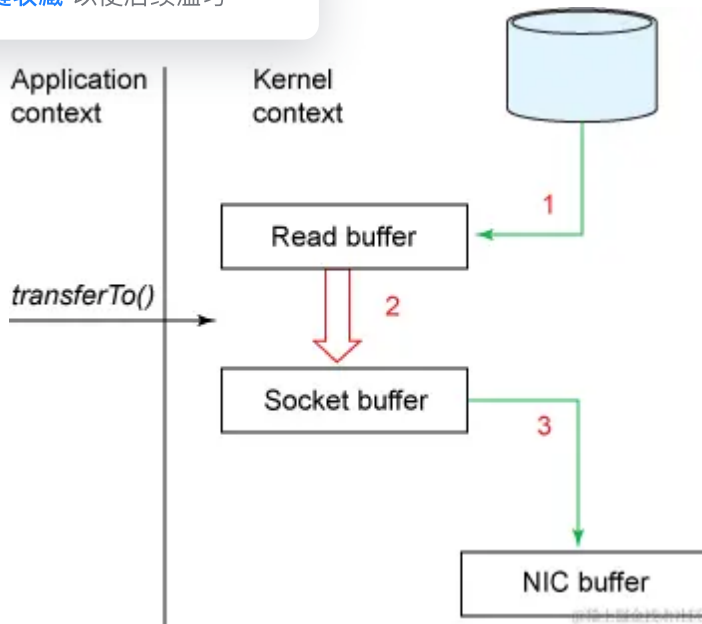
快速解读代码

立即体验

觉得还不错？

流程：

[一键收藏](#) 以便后续温习~



登录掘金领取礼包

更多登录后权益等你解锁

零拷贝的特点是 CPU 不全程负责内存中的数据读写。注意，零拷贝不是不进行拷贝，而是 CPU 不再全程负责数据在内存中，那么必须先通过某种方式拷贝到内存中，才能被转移，才能被 CPU 直接读取计算。

。但注
本身不
数据只有

Safetensors 优势

上面对比了不同模型权重存储格式，从中可以发现 Safetensors 主要优势有：

- 安全：使用 `torch.load()` 加载模型权重可能会执行被插入的（是不安全的），不过可以设置 `weights_only=False` 避免这个问题。

AI代码助手上线啦
选中代码，体验AI替你一键快速解读代码
立即体验
- 速度快：Safetensors 接口的速度比原生的 Pytorch 接口加载权重更快。
 - CPU 上加载提速的原因：通过直接映射文件，避免了不必要的复制（零拷贝）
 - GPU 上加载提速的原因：跳过不必要的CPU分配。

觉得还不错？
[一键收藏](#) 以便后续温习～

不加载整个文件的情况下查看文件的信息或者只加载文件中的部分张量而
我们有一个包含许多键和值对的大文件时，延迟加载非常重要。如果我们
可以单独加载单个键的值，它将提高内存效率并且速度更快，否则我们将不得不将完整文件
加载到内存中以检查任何键。

Safetensors 实践

安装

```
1 pip install safetensors
```



复制代码

保存与加载张量

保存张量：

```
1 import torch
2 from safetensors.torch import save_file
3
4 tensors = {
5     "embedding": torch.zeros((2, 2)),
6     "attention": torch.zeros((2, 3))
```



复制代码

登录掘金领取礼包

更多登录后权益等你解锁



```
7 }  
8 save_file(tensors, "model.safetensors")
```

加载张量：

▼ python

```
1 from safetensors import safe_open  
2  
3 tensors = {}  
4 # with safe_open("model.safetensors", framework="pt", device=0) as f:  
5 with safe_open("model.safetensors", framework="pt", device='cpu') as f:  
6     for k in f.keys():  
7         tensors[k] = f.get_tensor(k) # loads the full tensor given a key  
8 nprint(tensors)
```

觉得还不错？

[一键收藏](#) 以便后续温习~

```
tensor([[0., 0., 0.],  
        [0., 0.]], device='cuda:0'),  
tensor([[0., 0.],  
        [0., 0.]], device='cuda:0')}]
```

```
12 # [0., 0.]], device='cuda:0')}]
```



AI代码助手上线啦

选中代码，体验AI替你一键

快速解读代码

[立即体验](#)

仅加载部分张量（延迟加载）：

▼ python



复制代码

```
1 tensors = {}  
2 # with safe_open("model.safetensors", framework="pt", device=0) as f:  
3 with safe_open("model.safetensors", framework="pt", device='cpu') as f:  
4  
5     tensor_slice = f.get_slice("embedding")  
6     vocab_size, hidden_dim = tensor_slice.get_shape()  
7     print(vocab_size, hidden_dim)  
8     tensor = tensor_slice[:, :hidden_dim] # change the hidden_dim to load part of the te  
9     print(tensor)  
10 # 2 2  
11 # tensor([[0., 0.],  
12 #         [0., 0.]])
```

将PyTorch模型保存为Safetensors格式



登录掘金领取礼包

更多登录后权益等你解锁



制代码

▼ python

```
1 from torchvision.models import resnet18  
2 from safetensors.torch import load_model, sa  
3 import torch  
4
```



```

5 model_pt = resnet18(pretrained=True)
6
7 # 保存 state dict 为 safetensors格式
8 save_model(model_pt, "resnet18.safetensors")
9 # Instead of save_file(model.state_dict(), "model.safetensors")
10
11
12 # 加载没有权重的模型
13 model_st = resnet18(pretrained=False)
14 load_model(model_st, "resnet18.safetensors")
15 # Instead of model.load_state_dict(load_file("model.safetens
16
17
18 # 使用随机图像对初始模型和从safetensors中加载权重的新模型进行推理
19 img = torch.randn(2, 3, 224, 224)
20

```



AI代码助手上线啦

选中代码，体验AI替你一键

快速解读代码

立即体验

觉得还不错？

[一键收藏](#) 以便后续温习～

```

24
25 with torch.no_grad():
26     print(torch.all(model_pt(img)==model_st(img))) # tensor(True)
27

```

结语

本文简要介绍了模型权重存储新格式 Safetensors，它具备安全、加载速度快等多个优点；并且可以在 HuggingFace 上面看到越来越多的模型使用Safetensors格式进行存储。

参考文档：

- [bin和safetensors区别是什么？](#)
- [Safetensors：保存模型权重的新格式*](#)
- [github: safetensors](#)
- [huggingface: safetensors](#)
- [Safetensors: a simple, safe and faster way to store and distribute tensors.*](#)
- [safetensors torch api](#)
- [老pytorch格式转换成safetensors格式脚本](#)
- [safetensors与老pytorch格式速度对比*](#)



登录掘金领取礼包

更多登录后权益等你解锁



本文收录于以下专栏



大模型实践

专栏目录

大模型实践

89 订阅 · 80 篇文章



AI代码助手上线啦

选中代码，体验AI替你一键

快速解读代码

立即体验

上一篇 **大模型参数高效微调技术原理...**

下一篇 **大模型量化技术原理：FP8**

觉得还不错？

[一键收藏](#) 以便后续温习～



[登录 / 注册](#) 即可发布评论！

最热 **最新**



我是王大你是谁 NLP，算法，大模型 @宇宙尽头

6

9月前 点赞 评论

...

相关推荐

使用 LlamaIndex 框架搭建 RAG 应用基础实践

3.1k阅读 · 2点赞

Meta开源大模型LLama2部署使用保姆级教程，附模型对话效果

11k阅读 · 25点赞

人工智能大模型之开源大语言模型汇总（国内外开源项目梳理）

559阅读 · 1点赞

AI LLM 利器 Ollama 架构和对话处理流程解析

698阅读 · 7点赞



登录掘金领取礼包

更多登录后权益等你解锁

×

精选内容

每周AI论文速递 (250331-250404)

叶子的技术碎碎念 · 31阅读 · 0点赞

MCP 探索笔记：查找与管理你需要的 MCP

无责任此方_修行... · 44阅读 · 0点赞

半开源语音克隆神器 MegaTTS3：安装难、用起来更难？手把手从安装到使用

mortimer · 30阅读 · 0点赞



AI代码助手上线啦

选中代码，体验AI替你一键快速解读代码

立即体验

觉得还不错？

一键收藏 以便后续温习～

MCP服务

大模型 MCP：开启 AI 与现实世界的无缝交互革命

巫山老妖 · 97阅读 · 0点赞

为你推荐

生成式AI概览与详解

夏沫的梦 4月前 344 1 评论

OpenAI AIGC

AI 绘画基础 - 细数 Stable Diffusion 中的各种常用模型 【🧙 魔导士装备图鉴】

嘿嘿不务正业 1年前 1.4k 13 2

AIGC 人工智能 掘金·金...

深入理解vLLM引擎参数详解

路人与大师 1年前 2.6k 点赞 评论

算法

ComfyUI模型详解

Charlo 2月前 587 点赞 评论

人工智能 设计

魔乐社区新版本发布，多项功能升级等你体验

魔乐社区 2月前 62 点赞 评论

人工智能

LLM详解

夏沫的梦 5月前 207 1 评论

能 LLM



登录掘金领取礼包

更多登录后权益等你解锁



vLLM API 接口详解

路人与大师 9月前 2.4k 2 评论

算法

大模型存储选型 & JuiceFS 在关键环节性能详解

JuiceFS 5月前 39 点赞 评论

vLLM引擎参数详解 从运行日志观察vllm进行模型部署的过程

闫广庆 11月前 11k 1 评论



AI代码助手上线啦

选中代码，体验AI替你一键

快速解读代码

立即体验

开源医疗大模型排行榜: 健康领域大模型基准测试

HuggingFace 11月前 1.1k 8 评论

人工智能

欢迎 Mixtral - 当前 Hugging Face 上最先进的 MoE 模型

觉得还不错?

966 2 评论

人工智能 AIGC LLM

一键收藏 以便后续温习~

一、本地部署大模型

黯淡之星 7月前 1.0k 2 评论

LLM ChatGPT

本地私有化部署DeepSeek模型完整指南

颜淡慕潇 2月前 7.8k 5 4

人工智能

怎么使用Stable diffusion中的models

程序那些事 10月前 523 2 评论

人工智能 AIGC 后端

5分钟手把手系列(八): 本地通过Browser Use进行浏览器AI自动化操作

Batistuta 1月前 3.3k 1 评论

AIGC LLM



登录掘金领取礼包

更多登录后权益等你解锁

